

Scan your entire site - view full-site scan pricing | [View Pricing](#)



Enter domain, subdomain, (



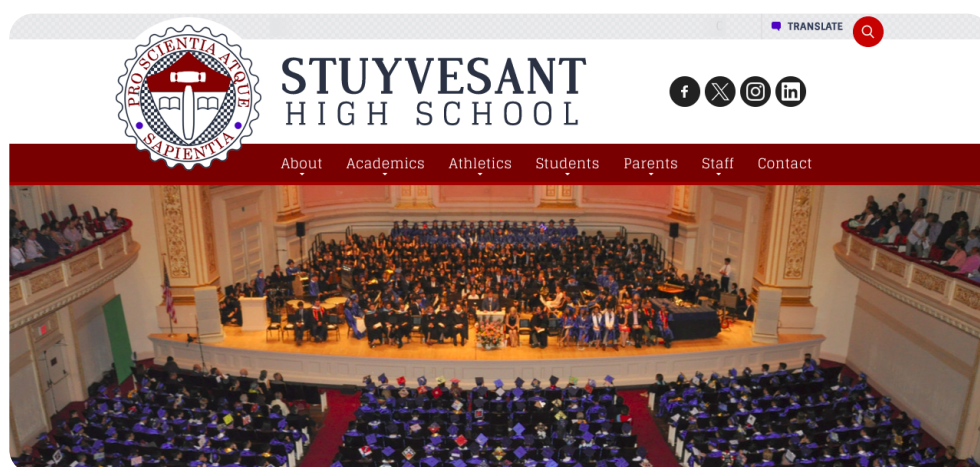
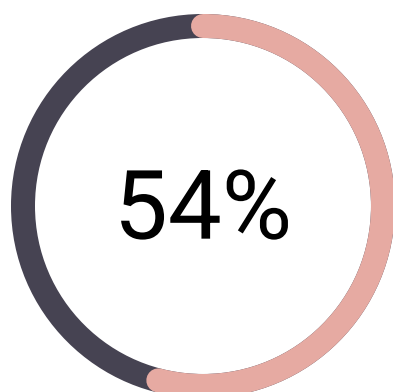
Web accessibility report



Page: stuy.enschool.org



Violations: 119



Book a demo

Visual Impairments (61%)
 Motor Impairments (67%)
 Cognitive (75%)
 Other Impairments (50%)



Significant accessibility barriers detected.



Your website poses serious challenges for many users. We recommend addressing the key issues first to improve access and avoid potential legal risks.

Perceivable [?]

Number of issues: 61



Titles built as text tags should be labeled as headings for assistive technology 7

Failed

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Elements that visually function as headings but are not coded with a heading tag should include a `role="heading"` attribute along with an appropriate aria-level (e.g., `aria-level="2"`), or be updated to use the correct semantic heading tag (e.g., `<h1>` to `<h6>`).

Why it matters

Assistive technologies use headings to navigate and understand page structure. If visual headings aren't coded properly, users may miss key information or struggle to navigate the content.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationship](#) →

Code Elements with Issues (7)

```
<span class="heading">Founded In</span>
```



Book a demo

```
"heading">Clubs and Activities</span>
```

```
<span class="heading">PSAL Sports Teams</span>
```



```
<span class="heading">AP Courses Offered</span>
```



```
<span class="heading">Average SAT Score</span>
```



```
<span class="heading">Average ACT Score</span>
```



```
<span class="heading">Electives Offered</span>
```



Pages must have proper landmark structure (one main, one banner, one contentinfo)

1

Failed

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Pages should contain exactly one main landmark (for primary content), exactly one banner landmark (for site header), and exactly one contentinfo landmark (for site footer). Banner landmarks are created by top-level `<header>` elements or explicit `role="banner"`. Contentinfo landmarks are created by top-level `<footer>` elements or explicit `role="contentinfo"`. Important: `<header>` and `<footer>` elements nested inside `<article>`, `<aside>`, `<main>`, `<nav>`, or `<section>` do NOT create banner/contentinfo landmarks—they are scoped to their containing element.

Why it matters

Landmarks are navigation shortcuts that screen reader users rely on to quickly jump between page regions. When a page has multiple banner or contentinfo landmarks, users cannot determine which is the actual site header or footer, forcing them to explore each one to find page-level navigation or copyright information. Missing or duplicate main landmarks prevent users from efficiently skipping navigation and jumping directly to the primary content. The WAI-ARIA specification explicitly states that authors SHOULD mark no more than one element with banner, contentinfo, or main roles per document to avoid this confusion.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



Book a demo

Code Elements with Issues (1)

Missing landmarks: main



Images conveying content embedded as background require proper labeling 48

Failed

We found elements and components on the website, which lack accessibility featur^{ss} related to this requirement.

Issue summary

When a background image conveys meaning and is used inside an interactive element that lacks an accessible name, the element should be assigned role="img" and given an accessible name using aria-label, aria-labelledby, or title.

Why it matters

Meaningful background images in interactive elements must be conveyed to assistive technologies. Without proper labeling, users with visual impairments may miss important content, reducing their ability to interact with or understand the purpose of the element.

Learn more in Tabnav Academy - [SC 1.1.1 Non-text Content](#) →

Code Elements with Issues (48)

```
<div class="nivo-slice" style="left: 0px; width: 1920px; height: 100%;
```



```
<div class="nivo-slice" style="left: 40px; width: 40px; height: 0px; c
```



```
<div class="nivo-slice" style="left: 80px; width: 40px; height: 0px; c
```



```
<div class="nivo-slice" style="left: 120px; width: 40px; height: 0px;
```



```
"nivo-slice" style="left: 160px; width: 40px; height: 0px;
```



Book a demo

```
<div class="nivo-slice" style="left: 200px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 240px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 280px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 320px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 360px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 400px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 440px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 480px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 520px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 560px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 600px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 640px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 680px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 720px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 760px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 800px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 840px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 880px; width: 40px; height: 0px;
```

[Book a demo](#)

```
<div class="nivo-slice" style="left: 920px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 960px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1000px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1040px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1080px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1120px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1160px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1200px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1240px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1280px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1320px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1360px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1400px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1440px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1480px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1520px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1560px; width: 40px; height: 0px;
```



Book a demo

```
<div class="nivo-slice" style="left: 1600px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1640px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1680px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1720px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1760px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1800px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1840px; width: 40px; height: 0px;
```



```
<div class="nivo-slice" style="left: 1880px; width: 40px; height: 0px;
```



SVG and Icon elements must include ARIA tags 5

Failed

We found elements and components on the website, which lack accessibility featuress related to this requirement.

Issue summary

Decorative SVG elements should be marked with `role="presentation"`, `role="none"`, or `aria-hidden="true"`. Meaningful SVG elements must include accessible names using `title/desc` tags, `aria-label`, `aria-labelledby`, or `title` attributes. Similarly, `<i>` elements used as icons must have a clear role (like `"img"`, `"graphics-document"`, or `"graphics-symbol"`) and accessible names via the same methods.

Why it matters

Screen readers rely on text alternatives to interpret visual icons and SVGs. Without accessible names, users with visual impairments may miss important cues or interactive elements, making it harder to understand or navigate the page effectively.



Book a demo

Academy - [SC 1.1.1 Non-text Content](#) →

Code Elements with Issues (5)

```
<svg width="24" height="24" viewBox="0 0 24 24" focusable="false" class="VIpjJd-ZVi9od-aZ2wEe" v
```



```
<svg width="24" height="24" viewBox="0 0 24 24" focusable="false" class="VIpjJd-ZVi9od-aZ2wEe" v
```



```
<svg width="24" height="24" viewBox="0 0 24 24" focusable="false" class="VIpjJd-ZVi9od-aZ2wEe" v
```



```
<svg width="24" height="24" viewBox="0 0 24 24" focusable="false" class="VIpjJd-ZVi9od-aZ2wEe" v
```



```
<svg xmlns="http://www.w3.org/2000/svg" class="VIpjJd-ZVi9od-aZ2wEe" v
```



Titles must contain text

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Headings (h1-h6 or role="heading") must contain text content. If a heading contains only graphical elements (images, SVGs, icons), those elements must have accessible names via alt attributes, aria-label, or aria-labelledby.

Why it matters

Screen readers announce headings to help users navigate page structure. Empty headings or headings with unlabeled graphics provide no useful information and create confusion about page organization.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



List items must be direct children of list containers



Book a demo



We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

List items (li elements or elements with role="listitem") must have a direct parent that is a list container (ul, ol, dl, or elements with role="list", role="menu", or role="menubar"). This ensures proper document structure and allows assistive technology to correctly interpret list hierarchy.

Why it matters

Screen readers announce list context and the number of items to users. When list items are not direct children of proper list containers, assistive technology cannot accurately convey the list structure, count, or navigation context. This breaks the semantic relationship and confuses users navigating through lists.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



Form controls must have programmatically determinable labels

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Form controls must be programmatically associated with their labels using <label> elements, aria-label, aria-labelledby, title attribute, placeholder attribute, or other accessible naming methods. This ensures assistive technologies can identify the purpose of each control.

Why it matters

Screen reader users cannot see visual labels next to form controls. Without a programmatic association between the label and the control, screen readers cannot announce what information should be entered, making forms difficult or impossible to complete independently. Note: While placeholders and title attributes can provide accessible names, they are not ideal substitutes for proper <label> elements as they have usability limitations.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



Book a demo

st include alternative text

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Every `<img>` element must include an alt attribute. For decorative images, use an empty alt value: `alt=""`. Images that convey meaningful information should have text that reflects their content. `<input type="image">` elements must always include alternative text, as they serve a functional purpose.

Why it matters

Assistive technology relies on alternative text to convey visual content to users with visual impairments. Without it, important information may be missed, making navigation difficult or incomplete.

Learn more in Tabnav Academy - [SC 1.1.1 Non-text Content](#) →



Landmarks of the same type must have unique accessible names

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

When multiple landmarks of the same type exist on a page (e.g., multiple `<nav>` elements or multiple `[role="navigation"]`), each must have a unique accessible name using `aria-label`, `aria-labelledby`, or `title` attributes. This allows users to distinguish between landmarks when navigating.

Why it matters

Screen reader users navigate pages by jumping between landmarks. When multiple landmarks of the same type exist without unique labels, users cannot distinguish between them - for example, "navigation" and "navigation" provide no context about which navigation region they are entering. Unique labels like "Main Navigation" and "Footer Navigation" enable users to efficiently navigate to the specific region they need.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



Book a demo



Submit buttons must be inside form elements

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Submit buttons (`button[type="submit"]` or `input[type="submit"]`) must be contained within a `<form>` element. A submit button's purpose is to submit its form owner - without being inside a form, the button has no programmatic relationship to any form and cannot perform its intended function.

Why it matters

Submit buttons that are not inside a form element are non-functional - they cannot submit any data. This creates confusion for all users, especially those using assistive technologies who may attempt to interact with a button that appears to be a submit button but does nothing. Users waste time filling out form fields only to find the submit button doesn't work. The missing structural relationship between the button and a form violates the programmatic relationships required by WCAG.

Learn more in Tabnav Academy - [SC 1.3.1 Info and Relationships](#) →



Object and Canvas elements must include ARIA tags



Book a demo



Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

<object> elements that contain visual content or media (such as video, audio, or images) must include labels using aria-label, aria-labelledby, or title. This is not required when the element does not display media. Similarly, <canvas> elements that present meaningful graphic content must include a role such as "img", "graphics-document", or "graphics-symbol", and be labeled using one of these methods.

Why it matters

Without appropriate labels, screen readers cannot detect or describe the content inside OBJECT or CANVAS elements, which may cause users with visual impairments to miss important information.

Learn more in Tabnav Academy - [SC 1.1.1 Non-text Content](#) →



Alternative text must be provided for Image Maps

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Image maps must include text alternatives for both the main image and each clickable area. The main element should have alt text describing the image's overall purpose. Each <area> element must include alt text that clearly identifies the function or destination of that region.

Why it matters

Without clear alt text for image maps and their regions, users who rely on screen readers cannot understand what each area does or where it leads. This creates a confusing or unusable experience for those navigating non-visually.

Learn more in Tabnav Academy - [SC 1.1.1 Non-text Content](#) →



Book a demo

able to 200% without loss of functionality

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

The viewport meta tag must not prevent users from scaling content. Avoid user-scalable=no, maximum-scale values less than 2.0, or minimum-scale values greater than 1.0. CSS properties like touch-action: none should not interfere with pinch-to-zoom gestures.

Why it matters

Users with visual impairments rely on zoom functionality to make content readable. Preventing scaling creates significant barriers and violates WCAG guidelines requiring content to be scalable up to 200% without loss of functionality.

Learn more in Tabnav Academy - [SC 1.4.4 Resize Text](#) →

Operable [?]

Number of issues: 3

✖ Page should have only one main heading 2

Failed

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Pages should contain only one h1 element or one role="heading" with aria-level="1". Multiple level 1 headings can confuse the document structure and make navigation difficult.

Why it matters

A single main heading helps users understand the primary purpose of the page and provides a clear starting point for navigation. Multiple h1 elements can create ambiguity about page hierarchy and make it harder for screen reader users to identify the main content.



Book a demo

Learn more in Tabnav Academy - [SC 2.4.6 Headings and Labels](#) →

Code Elements with Issues (2)

```
<h1>Pro Scientia Atque Sapientia</h1>
```



```
<h1><a href="/apps/news/" class="news-main-title">News <span>&amp;</span></h1>
```



Interactive elements are required to have a visible focus indicator for keyboard navigation

1

Failed

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Interactive elements must display a visible focus indicator—such as an outline or border change—when navigated to via keyboard. This ensures users can clearly see which element is currently focused.

Why it matters

Without visible focus indicators, keyboard users cannot tell which element they are interacting with, leading to confusion, missed actions, and a frustrating navigation experience—especially for users with motor impairments.

Learn more in Tabnav Academy - [SC 2.4.7 Focus Visible](#) →

Code Elements with Issues (1)

```
<button class="en-news-prev en-page-comp-arrow en-card-slider-button">
```



Interactive elements are required to be labeled for keyboard navigation

[Book a demo](#)

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Any element that looks and behaves like an interactive control must be accessible via keyboard using a tabindex of 0 or greater. Elements with tabindex="-1" are excluded when used for custom focus management.

Why it matters

Users who rely on a keyboard—such as people with visual or motor impairments—must be able to navigate and activate all interactive elements. If a control is not reachable by keyboard, it becomes completely unusable for these users.

Learn more in Tabnav Academy - [SC 2.1.1 Keyboard](#) →



Provide skip links for keyboard navigation

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Web pages should include skip links before repeated content (like headers or navigation menus) to let keyboard users jump directly to the main content. This improves navigation efficiency and reduces repetitive tabbing.

Why it matters

Without skip links, keyboard users must tab through every repeated section on each page, which can be time-consuming and frustrating. Skip links make navigation faster and more accessible for people with visual or motor impairments.

Learn more in Tabnav Academy - [SC 2.4.1 Bypass Blocks](#) →



Book a demo



Every page is required to include a TITLE element

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Each web page must have a descriptive and concise <title> element that clearly identifies the page's purpose, even when read out of context. A missing or empty <title>, or one that exceeds 100 characters, does not meet this requirement.

Why it matters

Visually impaired users using screen readers depend on clear, descriptive link text to understand where a link will take them.

Learn more in Tabnav Academy - [SC 2.4.2 Page Titled](#) →



Links must have descriptions that explain their purpose

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

All links must have accessible names that clearly describe their purpose or destination. This can be provided through visible text content, aria-label, aria-labelledby, title attributes, or alt text on child images.

Why it matters

Users of screen readers and other assistive technologies rely on link descriptions to understand where a link will take them. Without clear descriptions, users cannot make informed decisions about whether to follow a link, leading to confusion and poor navigation experience.

Learn more in Tabnav Academy - [SC 2.4.4 Link Purpose \(In Context\)](#) →



Book a demo



Use negative tabindex on hidden interactive elements to prevent keyboard interaction

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Interactive elements that are positioned off-screen for functionality should use `tabindex="-1"` to exclude them from the tab order. Without it, keyboard users may unknowingly navigate to off-screen elements, disrupting the logical focus flow.

Why it matters

When off-screen elements remain in the tab order, users can lose track of where they are on the page. This breaks the expected navigation flow, leading to confusion—especially for people using screen readers, keyboard navigation, or those with cognitive disabilities.

Learn more in Tabnav Academy - [SC 2.4.3 Focus Order](#) →



Form Control Accessible Name (Programmatically Determined) Must Match Visible Label

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

When a form control has both a visible label and an accessible name (via `aria-label`, `aria-labelledby`, etc.), the accessible name must include the exact text of the visible label to ensure consistency for all users.

Why it matters

Users with motor impairments who use voice control software (like "click Submit" or "click Email") rely on the visible label text being present in the accessible name. When the accessible name differs from or doesn't include the visible label, voice commands may fail, preventing these users from interacting with the form.



Book a demo

Tabnav Academy - [SC 2.5.3 Label in Name](#) →



Search input fields must have role="search" landmark on wrapper element

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Search input fields should have role="search" applied to one of their wrapping elements (such as a form or div container) to create a proper ARIA landmark. This landmark allows assistive technology users to quickly navigate to the search functionality.

Why it matters

Screen reader users can navigate pages efficiently by jumping between landmarks using keyboard shortcuts (e.g., pressing "L" to cycle through landmarks). Without role="search" on the wrapper element, users must tab through the entire page to find search functionality, which is time-consuming and frustrating. Landmarks enable quick access to key page sections including search.

Learn more in Tabnav Academy - [SC 2.4.1 Bypass Blocks](#) →



Popup windows and dialogs must trap keyboard navigation



Book a demo



Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Modal dialogs must keep keyboard focus within their boundaries while open. This prevents users from navigating to background content and ensures a logical, contained focus experience. Focus should remain inside the dialog until it is intentionally closed.

Why it matters

If focus is not trapped inside a modal, keyboard users can accidentally tab into content behind it, creating confusion and breaking the interaction flow. This can be especially disorienting for users with visual, motor, or cognitive impairments.

Learn more in Tabnav Academy - [SC 2.1.2 No Keyboard Trap](#) →

**Popup windows and dialogs must provide a keyboard-accessible exit option****Irrelevant**

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Modal dialogs, popups, and overlay windows must include a mechanism that allows keyboard users to close or exit them without using a mouse. This can be achieved through the Escape key, a visible close button that's keyboard accessible, or other keyboard shortcuts clearly indicated to users.

Why it matters

Keyboard users—especially those with visual or motor impairments, rely on clear, predictable navigation. If they can't exit a modal or popup, they may become stuck and forced to reload or abandon the page entirely, resulting in a poor and frustrating experience.

Learn more in Tabnav Academy - [SC 2.1.2 No Keyboard Trap](#) →



Book a demo

Understandable

Number of issues: 50

**Mark text in a different language using the lang attribute 2****Failed**

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Text written in a language different from the one specified in the `<html lang="...">` attribute must be explicitly marked with its own lang attribute. This ensures accurate interpretation by assistive technologies.

Why it matters

Without proper language tagging, screen readers may mispronounce content, and translation tools may not work correctly. This can confuse users and lead to misunderstandings, especially for those who rely on assistive tech to read or translate content.

Learn more in Tabnav Academy - [SC 3.1.2 Language of Parts](#) →

Code Elements with Issues (2)

```
<option value="tl">Filipino</option>
```



```
<span>Students and Staff of Stuy</span>
```

**Links that open in new windows should have clear indicators 47****Failed**

We found elements and components on the website, which lack accessibility features related to this requirement.

**Book a demo**

Links that open in new windows or tabs should be clearly identified so that users know they will be taken to

a new browsing context. This can be done using visual cues such as icons, labels (e.g., "opens in a new window"), or ARIA attributes.

Why it matters

Without clear indicators, users may be confused or disoriented when links unexpectedly open in new windows, especially users with cognitive disabilities or those using assistive technologies.

Learn more in Tabnav Academy - [WCAG Guidelines](#) →

Code Elements with Issues (47)

```
<a href="https://www.facebook.com/StuyNYC/" target="_blank" class="fb">
```



```
<a href="https://twitter.com/StuyNY" target="_blank" class="tw">Twitte
```



```
<a href="https://www.instagram.com/stuyvesanths_nyc/?hl=en" target="_b
```



```
<a href="https://www.linkedin.com/school/stuyvesant-high-school/" targ
```



```
<a href="https://www.facebook.com/StuyNYC/" target="_blank" class="fb">
```



```
<a href="https://twitter.com/StuyNY" target="_blank" class="tw">Twitte
```



```
<a href="https://www.instagram.com/stuyvesanths_nyc/?hl=en" target="_b
```



```
<a href="https://www.linkedin.com/school/stuyvesant-high-school/" targ
```



```
<a id="nmL6" tabindex="-1" href="http://stuy.entest.org/staff%20email9
```



```
<a id="nmL10" tabindex="-1" href="/apps/video/" target="_blank">Featur
```



```
<a id="nmL25" tabindex="-1" href="/apps/pages/index.jsp?uREC_ID=553128
```



```
<a id="nmL31-d3" tabindex="-1" target="_blank" href="https://stuy.ente
```



Book a demo

```
<a id="nmL31-d3" tabindex="-1" href="https://stuy.entest.org/School%20COU
```





































Book a demo



<a id="nmL65" tabindex="-1" href="http://stuycaliper.yolasite.com/" ta 

<a id="nmL66" tabindex="-1" href="https://www.stuyresearchclub.com/pre 

<a id="nmL68" tabindex="-1" target="_blank" href="https://stuy.enschoo 

<a id="nmL70" tabindex="-1" href="/apps/pages/index.jsp?uREC_ID=330259 

<a id="nmL71" tabindex="-1" href="https://stuy.entest.org/2025-2026%20 

<a id="nmL72" tabindex="-1" href="https://stuy-pa.org/" target="_blank 

<a id="nmL73" tabindex="-1" href="http://stuy.entest.org/staff%20email 

<a id="nmL79" tabindex="-1" href="http://outlook.com/owa/schools.nyc.g 

<a id="nmL80" tabindex="-1" href="https://talos.stuy.edu/" target="_b 

<a id="nmL81" tabindex="-1" href="https://login.jupitered.com/login/? 

<a href="http://stuy.libguides.com/c.php?g=239216&p=1590634" target 

Parents Associat 

<a href="http://stuy.enschool.org/stuycube/index.html" target="_blank" 

NYC DOE Webpage 

Stuyvesant Spectato 

Learn more about 



Book a demo

**Form controls must have visible labels or instructions 1****Failed**

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Each form control should have visible text that clearly describes its purpose. This can be a label, instruction text, or other visible indication of what the user should enter. Note that placeholders are not considered labels as they disappear while the user is typing.

Why it matters

Users with cognitive disabilities, learning disabilities, or memory impairments need visible labels to understand what information is required in each field. Without clear visible labels, users may enter incorrect information, abandon the form, or struggle to complete tasks successfully.

Learn more in Tabnav Academy - [SC 3.3.2 Labels or Instructions](#) →

Code Elements with Issues (1)

```
<select class=" skiptranslate " aria-label="Select translation language
```

**Keyboard navigation should not cause unexpected focus changes****Passed**

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Interactive elements should not automatically change the user's context (such as moving focus to another element) when they receive focus. Users should be able to navigate predictably through the interface. Context changes should only occur when the user explicitly activates a control.

**Book a demo**

Unexpected context changes can be disorienting and confusing for all users, but especially problematic for people using screen readers or those with cognitive disabilities. When focus jumps unexpectedly, screen reader users may lose track of their position on the page, and users with cognitive impairments may struggle to understand what happened or how to continue their task.

Learn more in Tabnav Academy - [SC 3.2.1 On Focus](#) →



The HTML element is required to include a lang attribute defining the page language

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

The <html> element is missing a lang attribute, which is required to define the default language of the page. This must be set before checking for inline language consistency.

Why it matters

Without a defined page language, screen readers may use the wrong pronunciation rules, making content harder to understand. This especially affects multilingual users and those relying on assistive technologies to interpret content accurately.

Learn more in Tabnav Academy - [SC 3.1.1 Language of Page](#) →

Robust [?]

Number of issues: 5



Avoid nesting interactive elements inside one another 1

Failed

We found elements and components on the website, which lack accessibility featur^{ess} related to this requirement.



Book a demo

Issue summary

One interactive element must not be placed inside another unless it's part of a valid pattern, such as <option> inside <select>. Nesting unsupported interactive elements can break accessibility for assistive technologies.

Why it matters

Improper nesting can confuse screen readers and disrupt expected keyboard behavior, making it difficult for users especially those with visual or motor impairments to understand, focus, or activate elements reliably.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →

Code Elements with Issues (1)

```
<a id="en-public-header-translate" en-google-translate="incl:de/German">
```



Interactive elements that do not have a semantic tag should be marked with an appropriate role attribute

4

Failed

We found elements and components on the website, which lack accessibility features related to this requirement.

Issue summary

Elements that appear and behave like interactive controls must have appropriate semantic roles (such as "button", "link", or "menuitem") or use correct HTML elements to convey their purpose to assistive technologies.

Why it matters

Without a defined role, screen readers and other assistive tools may not recognize that an element is interactive. This can confuse users, prevent them from performing actions, and reduce overall accessibility—especially for users with visual or cognitive impairments.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Book a demo

Code Elements with Issues (4)

```
"en-public-header-translate" en-google-translate="incl:de/German">
```



```
<div onclick="_enSearchPopupToggle()" tabindex="0" class="search-wrap
```



```
<h2 onkeyup="javascript:EduNetFeaturedAlbumCoverClick(event, '/apps/a
```



```
<li tabindex="0" aria-label="Students and Staff of Stuy album" onkeyd
```



Trigger elements that open submenus should use aria-haspopup to indicate submenu presence to assistive technologies

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

When a menu or list item contains a submenu, the clickable element that opens it (such as a button or link) must have the aria-haspopup attribute to inform assistive technology users that activating it will reveal additional options.

Why it matters

Screen reader users need to know when an item has a submenu before activating it. Without aria-haspopup, users may be surprised by unexpected navigation changes or miss nested content entirely. This is especially critical in navigation menus where submenus open on hover or click.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Elements that expand or collapse submenus should use aria-expanded to indicate their current state to assistive technologies

Passed

We found elements and components on the website with the required accessibility features correctly implemented.



Book a demo

When a list item or menu item contains a submenu, the interactive element that opens it (such as a button

or link) must have aria-expanded="true" when the submenu is visible and aria-expanded="false" when hidden. This value must update dynamically to reflect the current state

Why it matters

Screen reader users cannot visually see whether a submenu is expanded or collapsed. Without accurate aria-expanded state, users don't know if submenus are open, may activate already-expanded items thinking they're collapsed, or navigate away thinking there's no more content when submenus are actually open.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Interactive elements without content should be marked with ARIA attributes

Passed

We found elements and components on the website with the required accessibility features correctly implemented.

Issue summary

Interactive elements must have accessible names provided through visible text, aria-label, aria-labelledby, title attributes, or properly labeled child elements. This ensures their purpose is communicated to assistive technologies.

Why it matters

Without accessible names, screen readers cannot announce what an interactive element does. This creates confusion for users with visual impairments, making it difficult or impossible to understand or use critical functionality on the page.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Links using href='#' as buttons should be identified with role='button'

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website



Book a demo

Anchor elements using href="#" for actions like opening modals or triggering scripts should be given role="button" or replaced with actual <button> elements. This ensures correct semantics and improves support for screen readers and keyboard users.

Why it matters

Using anchor tags for non-navigation purposes can confuse assistive technologies and users. Screen readers may misidentify the element's purpose, and keyboard users may expect navigation instead of action, leading to a poor and inconsistent experience.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Hidden interactive elements should be excluded from screen readers using ARIA attributes

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Interactive elements that are visually hidden or positioned off-screen for styling purposes using CSS, and are not meant to be interacted with, should include aria-hidden="true" to exclude them from the accessibility tree.

Why it matters

Screen reader users may encounter off-screen elements that serve no functional purpose, leading to confusion and unnecessary navigation. Hiding these elements properly improves clarity and reduces cognitive load for users relying on assistive technologies.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Labels should not contain interactive elements other than the form control they label

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website



Book a demo

Issue summary

Form control labels (<label>) should not contain additional interactive elements such as links or buttons, other than the implicitly associated form control. Additional interactive elements make it difficult for users to activate the form input associated with the label.

Why it matters

When a label contains multiple interactive elements, it creates confusion about what will be activated when users click or tap. Screen reader users may not understand which element they are interacting with, and keyboard users may have difficulty navigating between the label's form control and other interactive elements. This particularly affects users with motor impairments who may accidentally activate the wrong element.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Listbox must contain option elements

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Elements with role="listbox" must contain one or more descendant elements with role="option". This establishes the proper parent-child relationship required for assistive technologies to understand and navigate the listbox structure.

Why it matters

Screen reader users rely on the proper ARIA structure to understand available options in a listbox. Without option descendants, assistive technologies cannot communicate the selectable items to users, making the listbox unusable for people with disabilities.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Option elements must not contain interactive descendants

Irrelevant



Book a demo

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Elements with role="option" can only contain text and must not include interactive elements such as links, buttons, or other focusable elements. All descendants of option elements are automatically treated as presentational, meaning they lose their semantic meaning and functionality.

Why it matters

When an option contains interactive elements like links, those elements lose their semantics and functionality for screen reader users. The accessibility API cannot represent semantic elements within an option, making nested interactive elements inaccessible. This prevents users from understanding or interacting with the content as intended.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Form should have an accessible name via ARIA tags

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

Form elements (<form> or [role="form"]) create form landmarks that must have programmatically determinable accessible names through aria-label, aria-labelledby, or title attributes. Per SC 4.1.2, user interface components must have names that can be programmatically determined so assistive technologies can identify them to users.

Why it matters

Form landmarks without accessible names cannot be properly identified by assistive technologies. Screen reader users navigating by landmarks will hear "form" without any context about the form's purpose, making it difficult or impossible to distinguish between multiple forms or understand what each form is for. This violates SC 4.1.2 because the component (form landmark) lacks a programmatically determinable name. Accessible names enable users to quickly identify and navigate to the specific form they need.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Summary elements must not contain interactive descendants



Book a demo

Irrelevant

We could not identify the necessary components or elements related to this requirement on the website

Issue summary

The <summary> element itself is interactive (it functions as a disclosure button for <details> elements). Placing interactive elements like links, buttons, or form controls inside a <summary> creates nested interactive contexts that break keyboard navigation and assistive technology interaction. Per SC 4.1.2, user interface components must be programmatically determinable and operable, but nested interactive elements within summary cannot be consistently accessed by keyboard or assistive technologies.

Why it matters

When a summary element contains interactive descendants, those nested elements will not be consistently accessible to people navigating by keyboard or using assistive technology. The summary element itself captures click and keyboard events to toggle the details element, preventing reliable access to nested interactive elements. This creates a situation where some users cannot activate links or buttons within the summary, or experience unpredictable behavior when trying to do so. The ambiguous focus management and event handling violates the requirement that all functionality be available from a keyboard and that interactive components have their role and state properly exposed.

Learn more in Tabnav Academy - [SC 4.1.2 Name, Role, Value](#) →



Full-Site Scan

Check the pricing for a full-site accessibility scan, covering every page.

[View Pricing](#)



[Book a demo](#)

WCAG Checker



Instantly detect issues with our free web accessibility checker.



Clear report to fixing your WCAG compliance gaps today.

[Start Free Scan](#)

Solutions

[Accessibility widget](#)[Accessibility monitor](#)[PDF remediation](#)[Litigation support](#)[Accessibility checker](#)[Contrast checker](#)

Resources

[Pricing](#)[FAQ](#)[Blog](#)[Integrations](#)[About](#)[Site map](#)

More

[Terms of Service](#)[Privacy Policy](#)[Cookie Settings](#)[Cookie Policy](#)[Partners](#)[Academy](#)

Contact Us

info@tabnav.com

14 NE 1st Avenue, Suite 1403 #1115,
Miami, FL 33132, United States

[Book a demo](#)[WCAG 2.2AA](#)[ADA Title III](#)

Section 508

EN301549

ACA/AODA

© 2025 Tabnav. All Rights Reserved

[Accessibility Statement](#)



[Book a demo](#)

